1 The Ghost Contributor

1.1 Product Requirements Document and Technical Architecture

Hacktoberfest Open-Source AI Challenge — Week 3: "Slow and Steady"

Document version: 1.0 · **Research checked:** October 21, 2026

Product type: Local-first developer maintenance assistant · **Status:** Proposal

> **One-line pitch:** Let a local AI agent quietly take care of small, real documentation chores—without sending source code to a cloud model, inventing work, or pretending a person did it.

1.2 1. Executive Summary

The Ghost Contributor is a Python background assistant that watches explicitly selected Git repositories for small, verifiable maintenance opportunities. It uses a locally running Ollama model to propose a focused code comment or documentation update, validates and previews the diff, then—only under a user-selected publishing policy—saves the change as a clearly attributed commit on a dedicated branch and optionally opens a pull request.

It is for developers who want less repo housekeeping, not a synthetic activity generator. The original "green contribution graph" joke captures a familiar bit of developer anxiety, but the product will not create empty commits, pad activity, randomize timestamps to imitate a human, or claim that AI-written changes were hand-authored. A contribution graph is not a quality metric, and GitHub's display rules do not guarantee that a legitimate commit appears on a profile. [3] [4]

The "lazy" value is real: the developer configures boundaries once, and the daemon does the boring discovery, drafting, validation, and safe delivery work. The human retains control of repository scope, allowed file types, publishing mode, model, schedule, and stop switch. The ghost handles chores; it does not forge a pulse.

1.3 2. Problem Statement

Developers maintain many repositories, but useful maintenance work is often small and easy to defer: explain a non-obvious function, clarify a configuration option, update a README example after a real change, or add a missing docstring. Each task is low-effort in isolation, yet context switching and review overhead make them accumulate.

A visible contribution graph can add pressure to perform activity for its own sake. That pressure creates a tempting but poor shortcut: unattended commits whose purpose is the graph rather than the repository. Such changes waste maintainer attention, risk misleading collaborators, and can degrade trust. GitHub's graph is criteria-based and subject to attribution, repository, branch, visibility, timing, and display rules; it is not a reliable proxy for meaningful work. [3] [4]

Product opportunity: reduce the cost of genuine maintenance by letting a local agent find and draft narrowly scoped improvements, while making review, attribution, and repository safety the default rather than afterthoughts.

1.4 3. Proposed Solution

A "set-it-and-forget-it" Python daemon scans only user-registered repositories and proposes one small, useful, prose-only maintenance change at a time. It operates through five explicit stages:

1. **Discover:** inspect repository configuration, status, supported files, and configured maintenance targets.
2. **Select:** choose a permitted target with a concrete documentation gap; skip when no worthwhile change is evident.
3. **Draft:** send the minimum relevant context to an Ollama model running on the same machine and request a structured proposal.
4. **Validate and preview:** enforce scope rules, apply a narrow syntax-aware transformation to a temporary copy, validate it, and display a diff.
5. **Deliver:** in default review mode, wait for approval. In an explicitly enabled unattended mode, publish only to a dedicated automation branch with a transparent identity and audit record. Never push directly to a protected/default branch.

"Deceptive automation" is not a product requirement. The intentionally cheeky name and "ghost" metaphor refer to chores getting done in the background, not hiding authorship or faking human activity. Scheduling jitter is permitted only as bounded operational load control; it must not be used to make commits look human.

1.4.1 Product goals

- Save developers time on small, real repository-maintenance tasks.
- Keep source context local by default and use an open-weight model through local Ollama inference.
- Preserve code behavior and formatting by limiting the MVP to comments and documentation.
- Make every proposed edit understandable, reversible, attributed, and auditable.
- Fail closed: when repository state, model output, parsing, validation, or push outcome is uncertain, do nothing and explain why.

1.4.2 Non-goals

- Generating empty commits or splitting one change into many commits to increase activity counts.
- Altering commit dates, simulating human work hours, or concealing AI involvement.
- Guaranteeing a green GitHub graph, eligibility for Hacktoberfest rewards, or maintainer acceptance.
- Refactoring application logic, editing dependencies, changing tests, or inventing undocumented behavior.
- Scanning repositories not explicitly configured by the user.
- Uploading code to a cloud LLM or silently enabling remote inference.

1.5 4. Users, Use Cases, and Success Measures

Primary user: a developer maintaining one or more personal or team repositories who wants small documentation chores handled with minimal context switching.

Primary use cases:

- Draft a missing or stale docstring for a selected Python function using existing code and tests as context.
- Explain an existing non-obvious configuration option without changing runtime behavior.
- Propose a README correction based on a verified source-of-truth file.
- Run in scheduled review mode and approve or reject a clear, bounded diff.

Success metrics (quality over volume):

- Accepted-proposal rate: proportion of reviewed proposals accepted without substantial rewriting.
- Change validity: proportion of accepted source edits that pass syntax and configured checks.

- Useful-change rate: proportion of published changes later retained by the repository owner or maintainer.
- Safety: zero writes outside allow-listed paths; zero commits that include pre-existing user changes; zero force-pushes; zero silent cloud-model calls.
- User effort: median time from notification to review/approval and reduction in deferred documentation tasks.
- Reliability: scheduled-run completion, duplicate-run prevention, and recoverable failure rate.

Anti-metrics: commit streaks, number of commits per day, graph color, randomized commit timing, and raw change volume. These reward activity rather than maintenance quality.

1.6 5. Functional Requirements

1.6.1 5.1 Repository registration and policy

- **P0:** Require explicit repository paths; canonicalize paths and reject paths outside the user-approved roots.
- **P0:** Per repository, configure include/exclude paths, permitted file extensions, schedule, timezone, model tag, validation commands, and publishing mode.
- **P0:** Default to dry-run plus review; never enable unattended publication during first-run setup.
- **P0:** Detect unsupported repository states, including bare repositories, detached HEAD for branch publishing, merge/rebase in progress, untracked or staged user changes, and an unexpected current branch. Stop without altering the worktree.
- **P1:** Support a dedicated worktree or temporary checkout for automation so concurrent developer edits do not collide with generated proposals.
- **P1:** Provide `pause`, `resume`, `run-once`, and `status` controls plus a global kill switch.

1.6.2 5.2 Discovery and candidate selection

- **P0:** Exclude secrets, generated files, vendored code, binaries, lockfiles, minified assets, and paths ignored by Git or explicitly excluded by the user.
- **P0:** Never treat “no change” as a failure. If no specific, evidence-backed maintenance opportunity is found, record a no-op and exit.
- **P0:** Select at most one focused change-set per repository per run. Do not split work into multiple commits to increase activity.
- **P1:** Allow a user-defined documentation backlog, such as specific functions or README sections, which takes precedence over speculative repo-wide discovery.
- **P1:** Consider age or low comment density only as candidate signals, never as proof that documentation is missing.

1.6.3 5.3 Generation and editing

- **P0:** Use Ollama’s local API or official Python client, with explicit host, model tag, request timeout, context limit, and non-streaming structured response unless streaming is needed for UI feedback. Ollama documents local chat/generation endpoints and JSON/JSON-Schema output; the application must still validate the result. [5] [6]

- **P0:** Provide only the smallest context needed: target source region, adjacent docstring/comment, relevant tests or configuration, and user-supplied style guidance. Do not send an entire repository by default.
- **P0:** Constrain the model to a structured proposal such as `{target, rationale, replacement_text, evidence, uncertainty}`. Reject missing fields, unexpected targets, executable code, imports, or content that exceeds configured bounds.
- **P0:** Restrict MVP changes to prose in Python comments/docstrings and selected Markdown documentation. No identifiers, expressions, control flow, or code behavior changes.
- **P0:** Show a unified diff and explicit rationale before approval or publication. Identify AI assistance in commit/PR text and product UI.
- **P1:** Support repository-specific style instructions, provided they cannot override safety policy or expand file permissions.

1.6.4 5.4 Validation

- **P0:** Parse supported Python files before and after; use AST to locate semantic docstrings and a lossless concrete-syntax transformation (such as LibCST) to preserve comments and formatting. Do not serialize whole source files with `ast.unparse()` to apply prose edits. [7] [8]
- **P0:** For Markdown, use a narrow, patch-oriented transformation and verify the changed region; preserve untouched bytes and line endings.
- **P0:** Validate that the candidate is syntactically valid, only approved spans changed, no unexpected file was created, and the edit is idempotent on a repeat run.
- **P0:** Run only configured, bounded, non-destructive checks. Compilation validates syntax but does not prove the new prose is factually correct or behavior-preserving; the proposal must cite its source context and remain reviewable. [7]
- **P0:** Write to a same-directory temporary file, retain a recoverable original, and replace atomically only after validation and approval. Fail closed on parse, encoding, newline, permission, or matching ambiguity. [9]
- **P1:** Add project-defined tests for before/after fixtures, Unicode, encoding cookies, CRLF/LF, final newlines, no-op cases, and malformed input.

1.6.5 5.5 Version control and delivery

- **P0:** Inspect the repository, worktree, branch, upstream, staged/unstaged/untracked files, and in-progress Git operations before writing or committing. Unexpected changes cause a safe stop.
- **P0:** Stage only the exact reviewed file list; inspect the staged diff before creating a commit. Use a message that describes the actual maintenance change and discloses AI assistance.
- **P0:** Use a new or configured automation branch, never a wildcard refspec, force-push, or direct push to a protected/default branch. GitHub branch protection and rulesets should remain authoritative. [10] [11]
- **P0:** Default delivery is a local diff or user-approved commit. Unattended mode requires explicit per-repository opt-in and writes only to the dedicated branch. Pull-request creation is optional and subject to repository policy.
- **P0:** Use existing Git credential helpers or SSH configuration. Never put credentials in URLs, prompts, command strings, config files, or logs. [12]

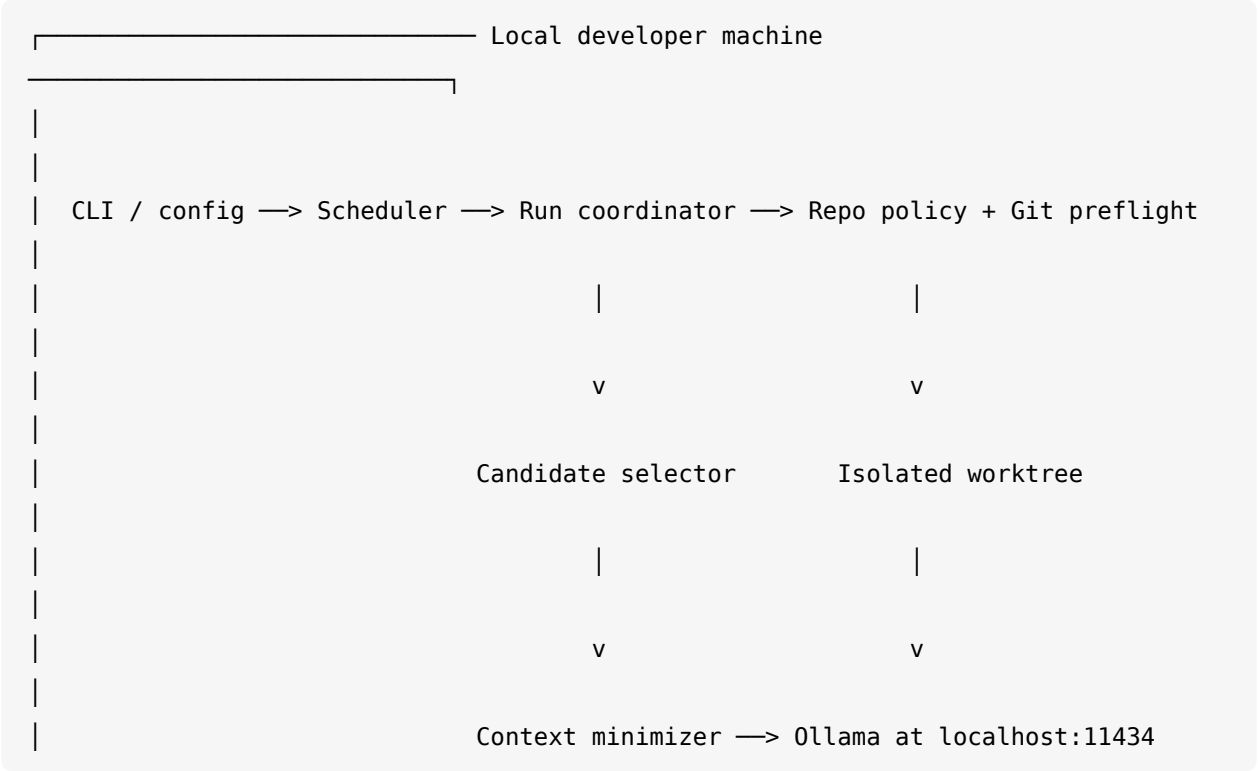
- **P0:** Use subprocess argument arrays without a shell when calling Git; set a working directory, timeout, and checked exit status. If using GitPython, check push result objects and raise on per-ref errors rather than assuming a returned call succeeded. [10] [13]
- **P0:** On push timeout or network failure, treat outcome as indeterminate: inspect the remote ref and reconcile before retrying. Never automatically escalate to force-push.
- **P0:** Write an append-only local run record with timestamp, repository identity, policy version, selected file, sanitized diff digest, model identifier, validation outcomes, approval identity/action, commit SHA, target ref, push result, and redacted error details.

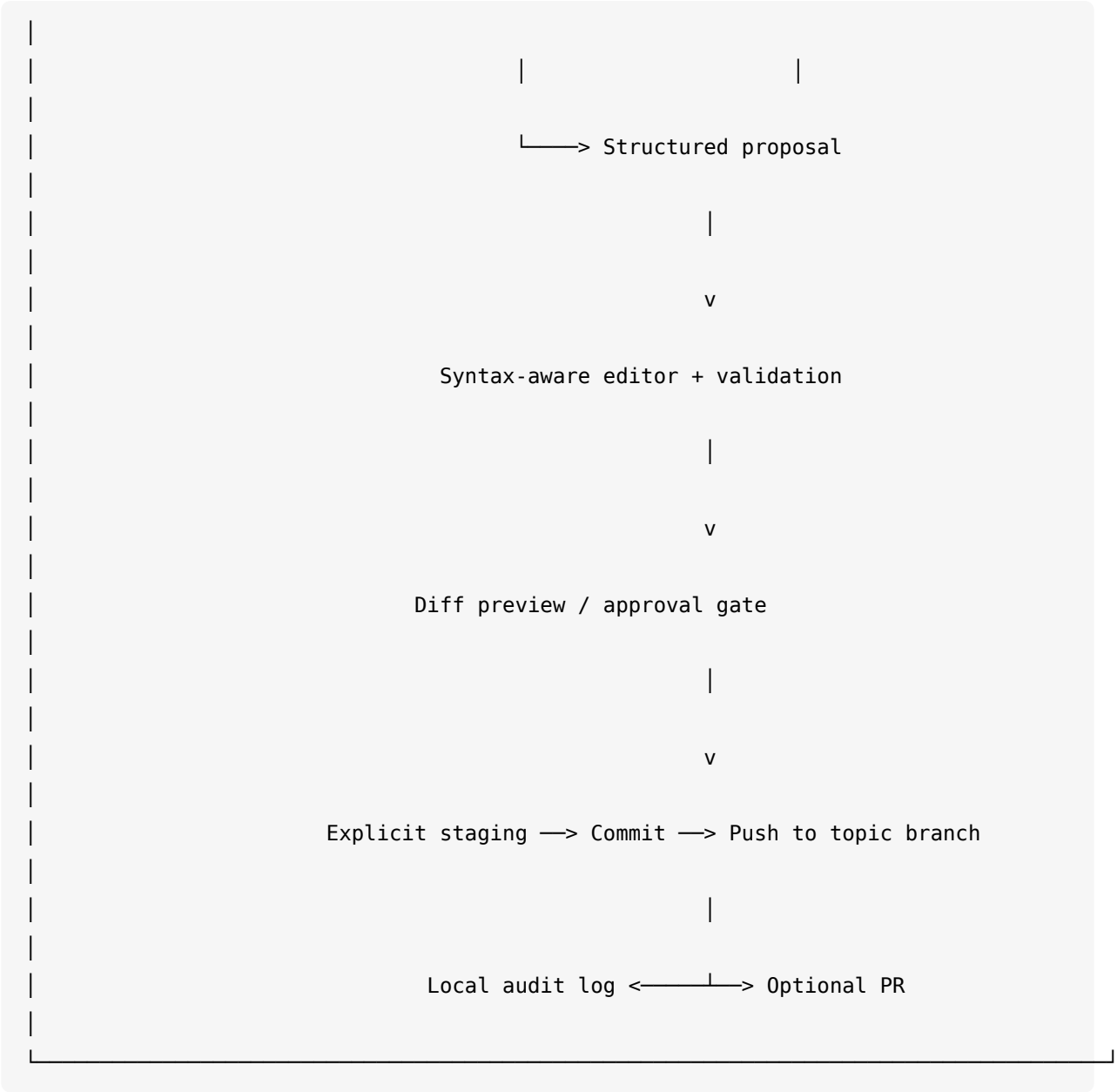
1.6.6 5.6 Scheduler

- **P0:** Use a user-visible fixed schedule and selected IANA timezone, with configurable quiet hours. Check quiet hours immediately before work begins.
- **P0:** Coalesce missed runs into at most one current pass; do not replay a backlog as a burst.
- **P0:** Prevent overlapping executions with a process lock and a stable task-period key; make retries idempotent and bounded.
- **P0:** Apply capped exponential backoff with bounded jitter to transient operational failures only. Jitter must stay inside the permitted schedule window and must never be used to imitate human activity. [14]
- **P0:** No-op when nothing useful is due. No fabricated timestamps, activity padding, or per-commit randomization.
- **P1:** Offer OS-native launch integration where feasible (for example, systemd user timers on Linux) while retaining application-level quiet-hour and coalescing checks. Named timezones and missed-activation behavior need explicit testing. [15]

1.7 6. Detailed Technical Architecture

1.7.1 6.1 Component view





The system is local-first, but Git hosting remains an external boundary: pushing a branch or opening a pull request necessarily communicates with the configured host. The daemon must show this clearly and avoid sending source to any model service other than the configured local Ollama endpoint.

1.7.2 6.2 Local inference engine

Ollama runs as a local model server, commonly at `http://localhost:11434`. Python can call the chat API with a model name and messages, or use Ollama's Python client. The implementation should discover installed models, verify the configured tag exists, and fail with a helpful local setup message when Ollama or the model is unavailable. [5] [6]

Model selection: expose a configured exact model tag rather than hard-coding "DeepSeek-Coder" or "Llama 3" as if each were one fixed model. The Ollama library lists materially different variants and context windows—for example, the DeepSeek-Coder family and original Llama 3 variants differ from later Llama 3.1/3.2 releases. Select based on task quality, context requirements, hardware capacity, latency, and the applicable model license. Displayed download size is not a guaranteed RAM requirement. [16] [17]

Prompt contract: ask for one documentation proposal grounded only in provided evidence. Require structured output with a bounded target path and replacement text. The prompt should explicitly prohibit code changes, invented facts, security-sensitive details, and claims not supported by the supplied source. Treat repository content as untrusted input; it must not be allowed to override system safety rules or execute tools.

Privacy contract: local inference avoids sending prompt content to a model provider when traffic remains on the local service, but this is not a blanket security guarantee. Bind to localhost, disable cloud features where appropriate, do not log source excerpts by default, and disclose that Git pushes still contact the remote host. Model license terms are separate from the Hacktoberfest challenge requirements. [18] [19]

1.7.3 6.3 The logic core: safe selection and editing

The first release supports a deliberately narrow surface: Python comments/docstrings and selected Markdown. The selector considers only configured, tracked, non-generated files. It can use heuristics—missing docstring, undocumented public option, mismatch between a documented example and a verified source—but must present the evidence behind each candidate. If it cannot establish a specific gap, it skips the file.

For Python, use AST to identify modules, classes, functions, and docstrings; do not use AST as a formatter. Python's AST serializer preserves equivalent syntax rather than exact source text and may discard comments or formatting. LibCST preserves comments and whitespace for targeted codemods; `tokenize` can support narrowly bounded comment edits but does not guarantee original spacing. [7] [8]

The transform should be deterministic after the proposal is accepted: map a validated target node to a precise CST node, replace only its prose field, render a candidate, and compare the resulting diff against an allow-list of intended spans. Refuse ambiguous matches, parse errors, overlapping edits, unsupported encodings, and source versions beyond the tested parser range. Use Tree-sitter only if later multi-language support justifies its additional grammar and coordinate complexity. [8]

Validation is layered: test the transformer on fixtures, parse/compile the candidate without executing it, confirm only allowed paths and spans changed, verify no-op/idempotence, and run user-configured checks with timeouts. A syntax check does not prove a comment is truthful; a cited evidence snippet and human review remain essential. Write via a temporary file and atomic replacement only after validation. [7] [9]

1.7.4 6.4 Version-control module

Git preflight is a hard gate, not a best-effort warning. The run coordinator records repository root, worktree path, current branch, upstream, remote, and working-tree status. It refuses to absorb existing staged, unstaged, or untracked user changes. In an isolated worktree, it creates or reuses a dedicated topic branch and rechecks the base revision before committing.

GitPython is suitable for repository inspection and commit operations; direct subprocess is also acceptable when invoked with a list of arguments, a fixed `cwd`, explicit timeout, and checked return code. Stage only the intended file paths; inspect the index diff; commit with a message based on the actual change, e.g. `docs: clarify cache timeout behavior (AI-assisted)`. Record the SHA and selected paths. Never disable hooks as a normal workaround. [10] [13]

Pushing uses an explicit remote and destination ref. A force update is never an automatic recovery path. Check per-ref push results and server response. Branch protection, required review/check policies, and

rulesets stay enabled. When a push fails, report the reason; if the outcome is unknown, reconcile remote state before any retry. [10] [11]

1.7.5 6.5 Scheduler and run state

The schedule is a human-readable maintenance window, not a human impersonation strategy. Store the timezone as an IANA identifier, enforce quiet hours in the application, and define daylight-saving-time behavior. If the machine is offline, coalesce the missed period into no more than one run. A nonblocking process lock prevents duplicate daemon instances; a stable task/period identifier prevents repeating completed work. [14] [15]

Use bounded jitter only for retry/load distribution, not for public timestamp shaping. Retry only transient failures, with finite attempts and timeouts. Permanent failures—invalid config, parse errors, policy rejection, missing model, or protected-branch rejection—should stop immediately. A scheduled pass that finds no justified edit records a no-op, not an empty commit.

1.7.6 6.6 Configuration and data boundaries

- **User configuration:** repository roots, path allow/block lists, model tag, schedule/timezone, quiet hours, validation commands, publication mode, and approval policy.
- **Local state:** lock files, task-period keys, proposal metadata, sanitized run logs, and optional encrypted secrets reference. Keep local state outside scanned repositories unless explicitly configured.
- **Model inputs:** minimal source excerpts and task instructions; no credentials, `.env` values, tokens, private keys, or unrelated repository files.
- **Network egress:** disabled for model inference by default; Git hosting traffic only for explicitly enabled branch push/PR operations.
- **Secrets:** rely on OS/Git credential managers or SSH agent; redact diagnostics before persistence.
- **Retention:** user-configurable proposal and audit-log retention; never store full source prompts by default.

1.8 7. Tech Stack

Layer	Proposed choice	Rationale
Runtime	Python 3.11+ (pin tested versions)	Cross-platform ecosystem, filesystem and process tooling
Local inference	Ollama + official Python client or local HTTP API	Local model serving, explicit model selection, structured output
Candidate parsing	Python `ast` for structure; LibCST for lossless Python edits	Separation of semantic location from source-preserving transformation
Git integration	GitPython for inspection and Git operations; `subprocess.run([...])` for bounded explicit commands	Clear repository APIs plus safe process invocation

Layer	Proposed choice	Rationale
Scheduling	OS-native timer where practical; application scheduler and lock for policy	Reliable background operation with application-level safeguards
Configuration	TOML or YAML with schema validation	Human-readable, versionable per-repository policy
Testing	`pytest` or `unittest`; golden diff fixtures; temporary Git repositories	Repeatable transformation and safety verification
Delivery	CLI first; optional local notification UI	Small MVP surface and transparent operator controls

Model note: DeepSeek-Coder and Llama 3 are candidate families, not a single interchangeable model. Pin a tested tag and record its digest, size, context limit, quantization, and license in each release's compatibility notes. "Open-weight" should not be casually equated with OSI-defined open-source AI; the exact model materials and license must be reviewed. [16] [17] [19]

1.9 8. User Flow: Installation to the First "Ghost Commit"

1. **Install:** the developer installs the Python CLI and verifies Git and Ollama are available. The setup wizard explains that local inference keeps prompts local only if cloud features/network exposure are disabled, while any Git push contacts the selected remote.
2. **Choose a model:** the user selects an installed exact model tag or explicitly pulls one. The tool displays approximate download size, configured context limit, and license link; it does not claim model downloads are cost-free in compute or electricity.
3. **Register a repository:** the user selects a local repository and allow-lists paths and file types. The tool scans configuration only; it does not silently crawl the home directory.
4. **Set policy:** review mode is enabled by default. The user chooses a schedule/timezone, quiet hours, validation commands, and whether future unattended publishing is allowed. Default branch pushes are prohibited.
5. **Preflight:** the daemon verifies repository status, branch/upstream, permissions, model availability, and local-only inference endpoint. If the tree contains unexpected changes, it stops and explains the issue.
6. **First run:** the user runs `ghost run --once --dry-run`. The tool identifies a concrete documentation gap, asks the model for a structured proposal, applies it to a temporary candidate, validates it, and displays the diff and rationale.
7. **Review:** the developer accepts, edits, or rejects the proposal. Rejection is recorded as feedback and the source remains unchanged.
8. **Commit:** after approval, the tool stages only the accepted paths and creates a transparent commit on a dedicated branch. The commit message describes the actual documentation work and marks it AI-assisted.

9. **Publish (optional):** if configured and authorized, the tool pushes to the topic branch and optionally opens a pull request. It reports the result and stores the SHA, remote/ref, validation summary, and sanitized log.
10. **Run unattended (optional):** only after explicit opt-in, later runs may repeat this bounded workflow. No useful candidate means no commit. A pause/kill switch is always available.

Example commands:

```
ghost init
ghost repo add /path/to/repository
ghost config set publish_mode review
ghost run --once --dry-run
ghost review latest
ghost publish latest
ghost status
ghost pause
```

1.10 9. Hacktoberfest Week 3 Fit and Compliance

Week 3's published "Slow and Steady" prompt asks for a new project built around open-source AI or open-weight models that saves effort or makes life easier, and asks entrants to explain why open innovation matters. The official DEV challenge window is October 19, 2026 at 9:00 AM PDT through October 25, 2026 at 11:59 PM PDT. The contest page requires a DEV post using its submission template/tags, a project demo, and code link; the rules call for a brand-new project created and completed during the entry period. Check the official rules and eligibility before submitting. [1] [2]

The idea's strongest theme fit is the real reduction of repetitive maintenance work. Local inference makes privacy and cost trade-offs part of the product design, rather than a decorative model call. But the product must not claim that it earns rewards by filling a contribution graph. The official Hacktoberfest 2026 FAQ says pull/merge requests no longer count toward rewards; reward activity and GitHub profile contributions are separate systems. [2] [3]

1.10.1 Why open-source AI and local-first matter here

- **Privacy:** source code may contain proprietary logic or sensitive context. Running inference locally can keep prompts off a model provider's servers, provided local-only configuration is maintained. Local mode is a design property to verify, not an automatic guarantee against all network egress. [18]
- **No per-request model API bill:** local inference avoids metered hosted-model charges for requests actually run locally. It still has hardware, electricity, storage, and maintenance costs. [20]
- **Inspectability and user choice:** developers can inspect the serving stack, select a model, and replace it; model terms and capabilities differ, so the project documents exact tags and licenses. Open weights alone do not prove compliance with OSI's Open Source AI Definition. [19]
- **Reproducibility:** pinning the tested model tag/digest and configuration lets maintainers understand which model generated a proposal, even when the model catalog later changes. [5] [16]
- **Appropriate boundaries:** local inference reduces one privacy risk but does not remove Git-hosting disclosure, inaccurate text, prompt injection in repository content, or license obligations.

Submission checklist: confirm current contest eligibility and exact terms; build and complete the project within the contest window if required; publish one DEV entry before the stated deadline using the provided template and tags; include a demo and source-code link; clearly explain where open-source AI is used and why it matters. Do not imply that commits themselves qualify for Hacktoberfest rewards. [1] [2]

1.11 10. MVP Scope, Phases, and Acceptance Criteria

1.11.1 MVP (P0)

- Python-only source transformations, plus selected Markdown files.
- Explicit repository registration and allow/block lists.
- Ollama local endpoint, installed-model check, exact model tag, bounded structured generation.
- One proposed prose-only edit per repo/run; dry-run/review default.
- Parser-backed editing, syntax validation, diff preview, clean-tree gate, explicit staging.
- Dedicated branch publishing only after opt-in; no direct default-branch pushes or force pushes.
- Fixed visible schedule, IANA timezone, quiet hours, no catch-up burst, overlap lock, local run log.
- Stop/pause/resume controls and safe no-op behavior.

1.11.2 Later phases (P1/P2)

- Broader language support through tested grammars and language-specific validators.
- Optional GitHub App/CLI integration for PR creation and status reporting.
- User-curated maintenance backlog, repository-specific style profiles, and opt-in quality analytics.
- Enterprise policy controls, signed policy configuration, and managed retention.

1.11.3 Release acceptance criteria

1. With a dirty worktree or unexpected staged changes, a run exits without modifying or committing anything.
2. A prompt-injection string in a README cannot change the tool's allowed target paths, run commands, or Git publication policy.
3. For all golden fixtures, an accepted comment/docstring edit changes only the intended prose and preserves formatting, encoding, and line endings.
4. Invalid model JSON, unsupported paths, parse errors, unsupported syntax, and ambiguous targets produce a logged safe no-op.
5. Re-running the same completed task period does not create duplicate edits or commits.
6. A push can target only the configured remote and dedicated branch; tests prove force-push and default-branch targets are rejected.
7. A failed or timed-out push is reconciled against remote state before any retry; no automatic force push occurs.
8. No model request leaves the local Ollama endpoint unless the user explicitly changes the privacy policy, and the UI shows that change before use.
9. A generated commit/PR identifies the work as AI-assisted and does not alter author/committer timestamps to imitate a human.
10. A no-op run creates no commit and does not attempt to manufacture activity.

1.12 11. Risks and Mitigations

Risk	Mitigation
Hallucinated or misleading documentation	Require evidence context, uncertainty field, diff review, and source citation in proposal metadata; default approval required
Prompt injection embedded in repository files	Treat repo content as untrusted data; no tool permissions in generation step; enforce path and command allow-lists outside the model
Accidental code change or formatting churn	Prose-only allow-list; lossless CST; candidate diff checks; golden fixtures; fail closed
Existing developer work captured in commit	Clean-tree gate; isolated worktree; explicit file staging and staged-diff review
Credential exposure	Existing credential helper/SSH agent; no tokens in prompts, URLs, logs, or config
Unauthorized direct push or branch rewrite	Dedicated branch only; server protections; explicit refspec; no force push; checked Git result
Duplicate or burst activity after downtime	Stable task-period key, process lock, coalesced catch-up, one change-set per run
Misleading public impression	Transparent AI attribution; no randomized author dates; meaningful changes only; no graph target metric
Model/license confusion	Record exact model tag and license; review model-specific terms; distinguish open-weight from OSI-defined open source
“Local” inference assumed private automatically	Localhost binding, disable cloud features, disclose any egress, test network behavior, minimize prompts
Contribution graph missing a commit	Do not promise display; ensure authentic attribution and eligible repo context only when relevant; explain that graph criteria and display timing are external to the product

1.13 12. Conclusion

The Ghost Contributor makes laziness useful: it automates the maintenance that developers actually intend to do, while keeping the edits small, local, transparent, and reversible. Its defining feature is not how often it commits. It is how confidently the developer can let it handle the boring parts and still trust what it changes.

The ghost can draft the docstring. The human still owns the truth.

1.14 References

1. Hacktoberfest Open-Source AI Challenge Week 3
2. Week 3 Contest Rules
3. GitHub Profile Contributions Reference
4. Troubleshooting Missing GitHub Contributions
5. Ollama Chat API
6. Ollama Structured Outputs
7. Python Abstract Syntax Trees
8. Why LibCST
9. Python os.replace
10. GitPython Tutorial
11. Managing a GitHub Branch Protection Rule
12. Caching GitHub Credentials in Git
13. Python subprocess
14. Timeouts, Retries, and Backoff with Jitter
15. systemd Timer Unit Configuration
16. Ollama DeepSeek Coder Model Library
17. Ollama Llama 3 Model Library
18. Ollama FAQ
19. Open Source AI Definition 1.0
20. Example of Usage-Based API Pricing